

Deadline. The homework is due **11:59pm, Tue 11/28, 2023**. You must submit your solutions (in pdf format generated by LaTeX) via GradeScope.

Late Policy. You have up to four grace days (calendar day) for the entire quarter. You don't lose any point by using grace days. If you decide to use grace days, please specify how many grace days you would like to use and the reason at the beginning of your submission.

Academic Integrity. You can discuss with your classmates or use online resources. In particular,

- **Collaboration Policy.** You can get help from the instructor, TAs, or your classmates, but only after you have thought about the problems on your own. **If you discussed with anyone, you need to acknowledge them in your report.**
- **Other sources.** It is OK to get inspiration (but not solutions) from books or online resources, again after you have carefully thought about the problems on your own. **If you use any reference or webpage, or discussed with anyone, you must cite it fully and completely in your report.**
- **AI Tools.** If you use ChatGPT or similar AI tools, you have to attach the full conversation to make it clear what help you obtained from them.

However, you **CANNOT**:

- share your report/code with anyone else,
- read other's report/code,
- copy anything from other source,
- get help from others without acknowledging them in your report, or use any relevant resources (e.g., online article/AI tools such as ChatGPT) without citing them.

Otherwise it will be considered cheating. We reserve the right to deduct points for using such material beyond reason. **In summary, you must write up your solution independently, and type your answers word by word on your own: close the book/notes/online resources when you are writing your final answers. Add all the citations and collaborators' names to the end.**

Write-up. Please use LaTeX to prepare your solutions. For all problems, **please explain how you get the answer** instead of directly giving the final answer, except for some special cases (will be specified in the problem). For all algorithm design problems, please describe using **natural language**. You can present pseudocode if you think that helps to illustrate your idea, but please do not only give some code without explanation. **In grading, we will reward not only correctness but also clarity and simplicity. To avoid losing points for hard-to-understand solutions, you should make sure your solutions are either self-explanatory or contains good explanations.** See some more details here <https://www.cs.ucr.edu/~yihans/teaching/214/S26/assignments/>

Bonus questions. Bonus questions are not required, and the points are not counted in the basic 100 points of the class. Most of the bonus questions are just slightly harder than the regular questions, but some can be extremely hard. In a nutshell, most of them are worthy trying, but don't be frustrated if you cannot solve *all* of them :) We will not provide too many hints/solutions for bonus problems, but feel free to discuss them with your classmates.

1 (4 pts + 2 bonus pts + 2 candies) Borůvka's's Algorithm

In this problem, let's design a parallel minimum spanning tree algorithm. The minimum spanning tree (MST) for a weighted undirected graph $G = (V, E)$, is a subgraph $T = (V, E')$ such that $E' \subseteq E$ and T is a tree that connects all vertices in V with minimum total edge weight. You probably have learned some MST algorithms in the sequential setting, such as Prim's and Kruskal's. Unfortunately, they are both hard to parallelize.

There's another famous MST algorithm called Borůvka's's algorithm. The sequential algorithm, proposed in 1926, is also the oldest MST algorithm we know of. This algorithm is a greedy algorithm and is reasonably simple. Given a graph $G = (V, E)$, we use $w_G(u, v)$ to denote the weight of edge $(u, v) \in E$. Assume all edge weights are distinct. Let $n = |V|, m = |E|$. The sequential algorithm is described as follows (you can also google for "Borůvka's", or read Wikipedia to learn more details).

Borůvka's MST Algorithm

1. For each vertex v in V , find the shortest edge incident on v and add it to MST T . Mark this edge as red. Use E_{red} to denote the set of all red edges.
2. Now we have a graph with some edges marked as red. We will contract this graph into a new graph $G' = (V', E')$ as described below:
 - (a) We first construct V' . Consider $G_{red} = (V, E_{red})$. Each vertex in G' is a connected component of G_{red} . That means that, as long as multiple vertices are connected by the red edges, we combine them into a super vertex in G .
 - (b) Then we consider E' . For two vertices $u', v' \in V'$, we add an edge between them if there exists $u \in u', v \in v'$, and $(u, v) \in E$. Recall that the vertices u' and v' are two red connected components of G . The weight of this edge is set to be

$$\min_{u \in u', v \in v', (u, v) \in E} w_G(u, v)$$

This means that, if originally, any two vertices in these two components are connected, we add an edge to connect u' and v' in G' . The weight of this edge is set to be the lightest among the crossing edges.

3. After contraction, we get a new graph G' , repeat the same process on G' , until we have selected $n - 1$ edges in the MST T .

Questions:

1. (0.5 pt) Prove that for any vertex $v \in V$, the lightest edge incident v must be included in the MST. You can consider using the *light-edge property* of the MST.
2. (0.5 pt) Prove that Borůvka's's algorithm is correct, i.e., all edges selected by the algorithm must all be in the MST of the graph.
3. (Bonus 1 pt + 1 candy) An important step in Borůvka's's algorithm is to contract the graph. Given a graph $G = (V, E)$, with no more than $O(n)$ red edges, show an algorithm that can contract the graph into a new graph G' as described in the above step 2. Your algorithm must run in $O(m)$ time and polylogarithmic span. If you choose not to work on this bonus question, you can assume the conclusion is true for the following questions.
4. (0.5 pt) Prove that $|V'| \leq |V|/2$. In other words, after contraction, the number of vertices shrink by at least a half after contraction.

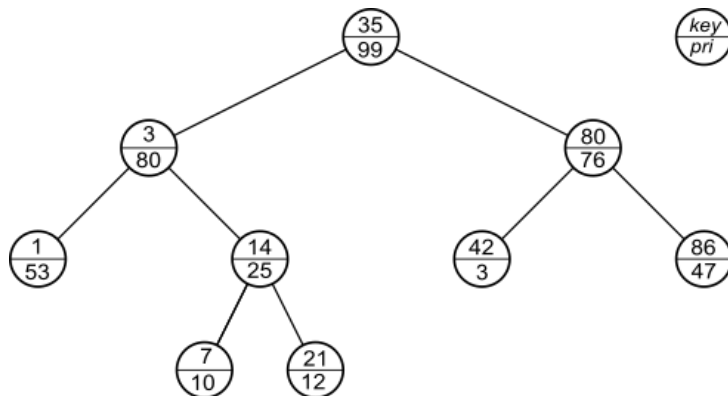
5. (1.5 pt) Let's try to parallelize Borůvka's based on its sequential version. As mentioned, you can assume there exists an efficient algorithm to contract the graph in Step 2, so our main task here is just to parallelize Step 1. How will you find and mark all the red edges in parallel and what are the work and span in this step? Please describe your algorithm in detail and analyze it.
6. (1 pt) Prove that the algorithm has $O(m \log n)$ work and polylogarithmic span. Again you can assume the conclusion in Problem 3 is true.
7. (Bonus 1 pt + 1 candy) Next let's prove a tighter bound. Prove the work of Borůvka's's algorithm is $\min(n^2, m \left(1 + \log \frac{n^2}{m}\right))$

2 Fun with Treaps (4 pts + 2 bonus pts + 2 candies)

In class we've seen some parallel tree algorithms. Although the general idea works for multiple balancing schemes, it still can be painful to deal with the rotations in the `join` algorithm. In this part, we'll try to implement these algorithms simply using a **treap**.

Treaps are randomized balanced binary trees. To maintain a set of keys using treaps, each key will be assigned a random **priority** (e.g, a random integer), or based on a random permutation of $1..n$. For simplicity, we assume all priorities are **unique** (otherwise you can break ties consistently). When constructing the tree, we will guarantee that the priority of a node is always higher than both of its children. In other words, all the keys on the tree forms a binary search tree, and all the priorities on the tree form a heap.

An example of a treap is shown in the following figure.



Source: http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=ALDS1_8_D

Basic Treap Properties and Constructing Treaps (1.5 pt)

We will first overview some basic properties of treaps, and discuss how to build a treap efficiently.

Answer the following questions:

1. (0.5 pt) Given the input keys and their priorities, what is the root of the tree?
2. (0.5pts) The height of a treap depends on the random priority. However, the probability distribution of the height of a treap is exactly the same as the probability distribution of the number of rounds in a **quicksort** algorithm, as long as we choose pivots in the quicksort uniformly at random. Please show the equivalence between the height of a treap and the number of rounds in a quicksort.
(This tells us that, the height of a treap is $O(\log n)$ with high probability. As a result, treap is also a “balanced” tree.)

3. (0.5pts) Based on the answers for the previous problems, given a list of n keys with their priorities, design an algorithm to construct a treap in $O(n \log n)$ work and $O(\log^2 n)$ span with high probability. Note that the n keys are not necessarily sorted.
4. (bonus, 1pt + 1 candy) If the keys are given in a sorted order, can you design an algorithm to construct a treap in $O(n)$ work and $O(\log^2 n)$ span with high probability?

Joining and Splitting Treaps (1 pt)

Joining two treaps $\text{join}(T_L, k, T_R)$ can be very easy if k has a higher priority than all keys in T_1 and T_2 . Then this join only need to connect T_L and T_R using k , in a constant time. We call such a join invocation a **direct join**.

Answer the following questions:

5. (0.5pts) In the **split** algorithm, show that when a **join** algorithm is called, it is always a direct join.
6. (0.5pts) Show that the cost of **split** on a treap is $O(\log n)$ with high probability.
7. (Bonus, 1pts + 1 candy) Describe a **general join** algorithm on two treaps and a key k with priority p_k . Now you **cannot** assume that p_k is higher than the priorities of all keys in the input trees.

Union/intersection/difference on two treaps (1.5 pts)

Since we can implement a **split** and a direct join on treaps, we can also implement the **union** algorithm. Recall the algorithm:

```
union(T1, T2) {
  if T1 is empty return T2;
  if T2 is empty return T1;
  k = key at the root of T2;
  L2 = the left subtree of T2;
  R2 = the right subtree of T2;
  (L1, b, R1) = split(T1, k);
  in parallel:
    TL = union(L1, L2);
    TR = union(R1, R2);
  return join(TL, k, TR);
}
```

Answer the following questions:

8. (1 pt) Now that we know the **split** operation only calls direct joins, we still need to guarantee the last join at the last line is a direct join. Please describe how you can guarantee the **join** at the last line is always a direct join by slightly modifying the algorithm.
9. (0.5 pt) Does the same modification in the above question work also for **intersection** and **difference** algorithms? If so, please describe the algorithms. If not, explain the reason.